

Travaux Pratiques 3

Cryptographie appliquée à la Blockchain

Clés, Signatures et Portefeuilles

Abdoullah Ichou
EPSI — Auxerre

19 mai 2026

Durée : 1h00 **Groupes :** Individuel ou binôme

Prérequis : Python 3.8+ et les bibliothèques `ecdsa` et `hashlib`

Installation : `pip install ecdsa`

Objectifs pédagogiques :

- Générer une paire de clés asymétriques (courbe elliptique SECP256k1)
- Signer une transaction avec une clé privée
- Vérifier une signature avec la clé publique correspondante
- Dériver une adresse de portefeuille par double hachage (SHA-256 + RIPEMD-160)
- Comprendre pourquoi une transaction ne peut pas être répudiée ni falsifiée

1 Mise en contexte

1.1 Le problème de l'identité sans tiers de confiance

Dans un système bancaire classique, la banque *authentifie* votre identité : elle sait que c'est vous qui avez signé un virement. Dans une blockchain, il n'y a **aucun tiers de confiance**. La cryptographie asymétrique résout ce problème.

1.2 Cryptographie à courbe elliptique (ECC)

Bitcoin utilise la courbe **SECP256k1**. Le principe :

- **Clé privée** (k) : un nombre aléatoire de 256 bits. *Secret absolu.*
- **Clé publique** ($K = k \cdot G$) : calculée à partir de la clé privée et d'un point générateur G sur la courbe. *Partageable publiquement.*
- **Sens unique** : on peut calculer K depuis k , mais il est **computationnellement impossible** de retrouver k depuis K (problème du logarithme discret sur courbe elliptique).

1.3 Signature numérique (ECDSA)

Pour "signer" un message m :

1. Alice calcule le hash de la transaction : $h = \text{SHA-256}(m)$
2. Alice utilise sa *clé privée* pour générer une signature $\sigma = \text{ECDSA_sign}(h, k)$
3. Elle publie (m, σ) sur le réseau

4. Tout le monde peut vérifier : $\text{ECDSA_verify}(\sigma, h, K_{\text{alice}}) = \text{True}$

Propriété clé : Seule Alice peut produire σ , mais *tout le monde* peut vérifier que σ est valide avec K_{alice} .

1.4 Adresse de portefeuille

L'adresse Bitcoin se dérive de la clé publique :

$$\text{Adresse} = \text{Base58Check}(\text{RIPEMD-160}(\text{SHA-256}(\text{clé publique})))$$

Dans ce TP, nous simplifions cette dérivation à : $\text{SHA-256}(\text{SHA-256}(\text{clé publique}))$.

Sécurité pratique : Ne jamais partager sa clé privée. Elle équivaut aux fonds associés à l'adresse. Dans Bitcoin, *il n'y a pas de « mot de passe oublié »*.

2 Code de démarrage

Installation préalable : Exécutez dans votre terminal : `pip install ecdsa`

Fichier : `tp3_starter.py`

```

1  #!/usr/bin/env python3
2  """
3  TP 3 --- Cryptographie appliquée : Code de démarrage
4  Complétez les parties # TODO dans l'ordre numérique.
5  """
6
7  import hashlib
8  from ecdsa import SigningKey, VerifyingKey, SECP256k1, BadSignatureError
9
10
11  # =====
12  # PARTIE 1 : Génération des clés
13  # =====
14
15  def generate_keypair():
16      """
17      Génère une paire de clés ECDSA sur la courbe SECP256k1.
18      (La même courbe utilisée par Bitcoin et Ethereum)
19
20      Returns:
21          private_key -- objet SigningKey (clé privée)
22          public_key   -- objet VerifyingKey (clé publique)
23      """
24      # TODO 1 : Générer une clé privée avec SigningKey.generate(
25      #           curve=SECP256k1)
26      #           En déduire la clé publique avec private_key.
27      #           Retourner (private_key, public_key)
28      pass
29
30  # =====
31  # PARTIE 2 : Signature d'une transaction
32  # =====

```

```

33
34 def sign_transaction(private_key, transaction: str) -> bytes:
35     """
36     Signe une transaction avec la clé privée.
37
38     Args:
39         private_key -- objet SigningKey
40         transaction -- chaîne de caractères décrivant la transaction (ex: "
41             Alice->Bob:5BTC")
42
43     Returns:
44         signature (bytes) -- la signature numérique de la transaction
45     """
46     # TODO 2 :
47     # 1. Encoder la transaction en bytes : transaction.encode()
48     # 2. Signer avec private_key.sign(message_bytes)
49     # 3. Retourner la signature
50     pass
51
52 # =====
53 # PARTIE 3 : Vérification de la signature
54 # =====
55
56 def verify_transaction(public_key, transaction: str, signature: bytes)
57     -> bool:
58     """
59     Vérifie qu'une signature est valide pour une transaction donnée.
60
61     Args:
62         public_key -- objet VerifyingKey (clé publique de l'expéditeur)
63         transaction -- la transaction dont on vérifie la signature
64         signature -- la signature à vérifier (bytes)
65
66     Returns:
67         True si la signature est valide, False sinon.
68     """
69     # TODO 3 :
70     # Utiliser public_key.verify(signature, transaction.encode())
71     # Cette méthode lève BadSignatureError si la signature est
72     # invalide.
73     # Capturer l'exception et retourner False dans ce cas.
74     pass
75
76 # =====
77 # PARTIE 4 : Dérivation d'adresse de portefeuille
78 # =====
79
80 def derive_address(public_key) -> str:
81     """
82     Dérive une adresse de portefeuille depuis la clé publique.
83
84     Implémentation simplifiée (sans Base58Check) :
85     adresse = SHA-256(SHA-256(clé publique en bytes)).hexdigest()[:40]
86
87     La vraie adresse Bitcoin applique aussi RIPEMD-160 et Base58Check.

```

```

87
88     Args:
89         public_key -- objet VerifyingKey
90
91     Returns:
92         adresse (str) -- 40 caract\`eres hexad\`ecimaux
93     """
94     # TODO 4 :
95     # 1. Extraire les bytes bruts de la cl\`e publique : public_key.
96         to_string()
97     # 2. Appliquer SHA-256 une premi\`ere fois
98     # 3. Appliquer SHA-256 une deuxi\`eme fois sur le r\`esultat
99     # 4. Retourner les 40 premiers caract\`eres hexad\`ecimaux
100     pass
101
102 # =====
103 # PARTIE 5 : Simulation d'une transaction Bitcoin
104 # =====
105
106 class Wallet:
107     """
108     Portefeuille simplifi\`e avec cl\`e priv\`ee, cl\`e publique et
109     adresse.
110     """
111     def __init__(self, name: str):
112         self.name = name
113         # TODO 5 : Appeler generate_keypair() et stocker les r\`esultats
114         #           D\`eriver l'adresse avec derive_address()
115         self.private_key = None
116         self.public_key = None
117         self.address = None
118
119     def __repr__(self):
120         return (f"Wallet({self.name}, address={self.address[:16] if self
121             .address else 'N/A'}...)")
122
123 if __name__ == "__main__":
124     print("=" * 60)
125     print("TP3 --- Cryptographie appliqu\`ee")
126     print("=" * 60)
127
128     # Cr\`eation de deux portefeuilles
129     alice = Wallet("Alice")
130     bob = Wallet("Bob")
131
132     print(f"\nAlice : {alice}")
133     print(f"Bob : {bob}")
134
135     # Transaction d'Alice vers Bob
136     transaction = f"Alice ({alice.address[:8] if alice.address else
137         '?'}...) -> Bob : 5 BTC"
138     print(f"\nTransaction : {transaction}")
139
140     # Signature par Alice
141     signature = sign_transaction(alice.private_key, transaction)
142     if signature:

```

```

141     print(f"Signature (hex) : {signature.hex()[:40]}...")
142
143     # V\`erification
144     valid = verify_transaction(alice.public_key, transaction, signature)
145     print(f"\nV\`erification avec cl\`e publique d'Alice : {valid}")
146
147     # TODO 6 (Exercice 3) : Tentative de fraude
148     # V\`erifiez la m\`eme signature avec la cl\`e publique de Bob
149     # Que se passe-t-il ?

```

3 Exercices

Exercice 1 — Compléter le squelette [TODO 1 à 5]

Complétez les 5 premières parties du code. Exécutez `python tp3_starter.py`.

- Observez la longueur de la clé publique et de la signature (en octets).
- Chaque exécution génère une nouvelle paire de clés : pourquoi ?

Exercice 2 — Propriétés de la signature

1. Modifiez **légèrement** la transaction (un seul caractère) et vérifiez à nouveau. Que se passe-t-il ?
2. La signature change-t-elle si vous signez deux fois le même message ? Pourquoi (ou pourquoi pas) ?
3. Quelle est la longueur en octets de la clé privée ? De la clé publique ? De la signature ? Complétez le tableau :

Donnée	Longueur (octets)
Clé privée (<code>.to_string()</code>)	
Clé publique (<code>.to_string()</code>)	
Signature	
Adresse (hexadécimale)	

Exercice 3 — Tentative de fraude [TODO 6]

1. Créez deux portefeuilles : Alice et Bob.
2. Alice signe la transaction "Alice -> Bob : 5 BTC".
3. Tentez de vérifier cette signature avec la clé publique de **Bob** (à la place d'Alice).
4. Que retourne `verify_transaction` ? Pourquoi ?
5. **Question de réflexion** : Si Mallory intercepte la transaction signée et essaie de la modifier pour "Alice -> Mallory : 5 BTC", la signature sera-t-elle toujours valide ? Pourquoi ?

Exercice 4 (Bonus) — Adresse Bitcoin réelle

Améliorez `derive_address` pour qu'elle applique également **RIPEMD-160** entre les deux SHA-256 (comme Bitcoin le fait réellement) :

$$\text{adresse} = \text{SHA-256}(\text{RIPEMD-160}(\text{SHA-256}(\text{clé publique})))$$

Utilisez `hashlib.new('ripemd160')`. Comparez la longueur obtenue.